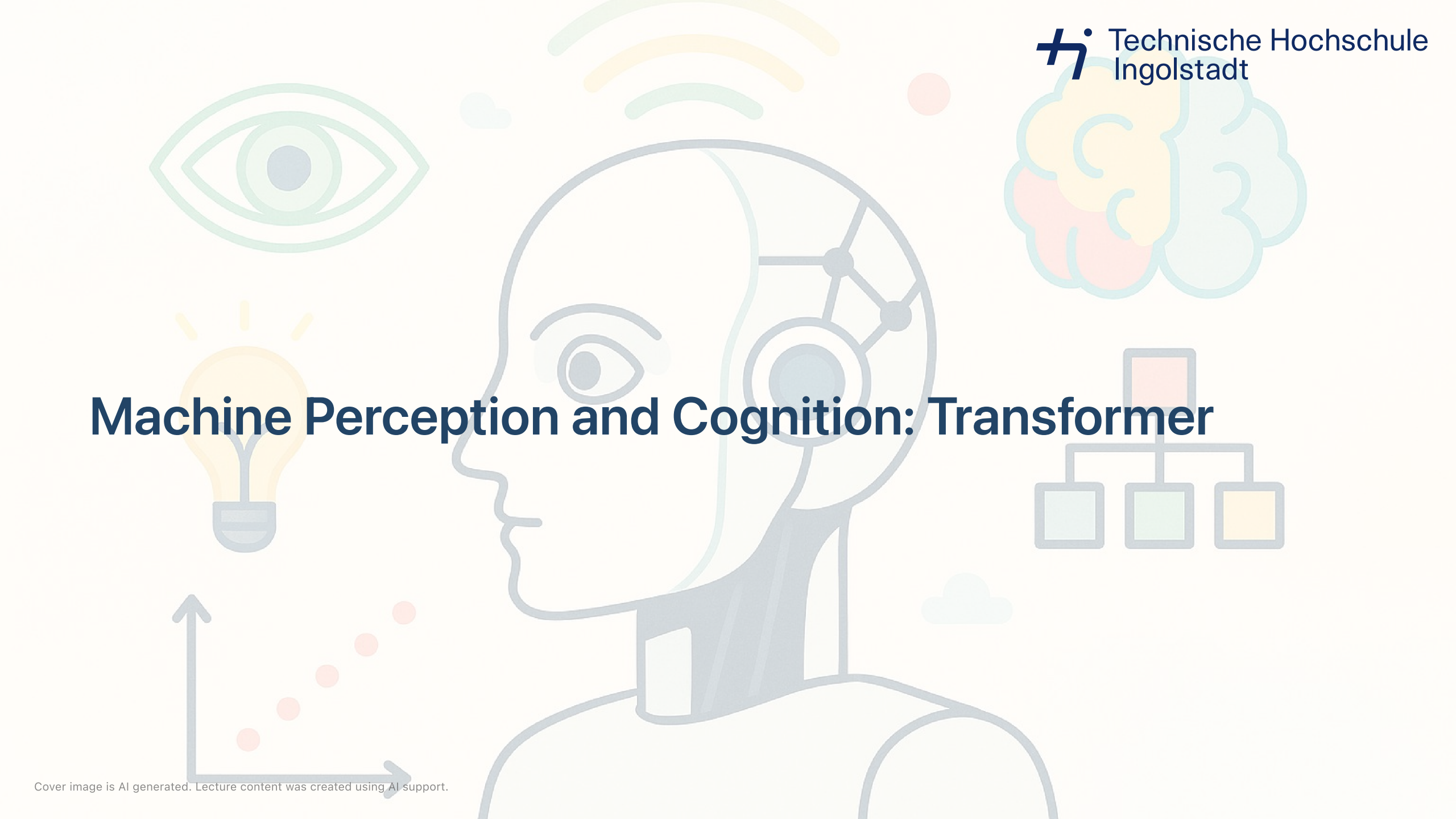
Technische Hochschule Ingolstadt

Machine Perception and Cognition: Transformer



Cover image is AI generated. Lecture content was created using AI support.

From RNNs to Transformers

Recurrent neural networks process sequences **step by step** — each hidden state depends on the previous one:

- **Sequential bottleneck**: processing of step t must wait for step $t-1$ — time steps must be computed sequentially during training
- **Vanishing gradients**: long-range dependencies are hard to capture even with LSTM
- **Fixed context**: the hidden state has limited capacity to carry information over very long sequences

From RNNs to Transformers

Transformer (Vaswani et al., 2017) addresses all three limitations:

- Processes the **entire sequence in parallel** using self-attention
- Connects **any two positions** directly — no intermediate steps
- Scales effectively to large models through parallel processing of sequence positions and efficient GPU utilization

Key Ideas of the Transformer

The Transformer is built on three fundamental ideas:

1. **Token embeddings** — represent discrete symbols as dense continuous vectors
2. **Positional encoding** — inject order information, since attention without positions is permutation-equivariant
3. **Multi-head self-attention** — each position directly attends to all others in parallel

These ideas are combined in stacked **encoder** and **decoder** blocks, each with residual connections and layer normalization, forming a flexible and highly scalable architecture.

What is Tokenization?

Before a neural network can process text, raw characters must be converted to discrete **tokens**:

$$\text{raw text} \xrightarrow{\text{tokenizer}} \text{sequence of token IDs} \xrightarrow{\text{embedding}} \text{sequence of vectors}$$

A **token** is the basic unit of text the model sees. It can be:

- A full word: `"transformer"` → 1 token
- A subword unit: `"transformer"` → `["transform", "er"]` → 2 tokens
- A single character

Most modern large language models use **subword tokenization**, which balances vocabulary size with coverage of rare and unseen words.

Word-Level vs. Subword Tokenization

Property	Word-level	Subword (BPE)
Vocabulary size	Very large (millions)	Controlled (30k–100k)
Unknown words	Frequent	Rare
Morphology	Ignored	Partially captured
Efficiency	Low for rare words	High

Problem with word-level: `"unhappy"`, `"unhappiness"`, `"unhappily"` are three unrelated tokens — the shared stem `"happy"` and prefix `"un"` are lost.

Subword tokenization decomposes words into reusable pieces: `["un", "happy", "ness"]` — shared morphemes are represented by shared tokens.

Byte Pair Encoding (BPE)

Byte Pair Encoding (Sennrich et al., 2016) is a widely used subword tokenization algorithm:

Algorithm:

1. Start with a character-level vocabulary
2. Count all adjacent symbol pairs in the training corpus
3. Merge the most frequent pair into a new symbol
4. Repeat until the vocabulary reaches the target size

Typical subword vocabularies used in practice range from **30,000 to 100,000 tokens**, balancing coverage with model size.

The Vocabulary

The tokenizer produces a **vocabulary** $\mathcal{V}$ — a finite set of tokens with size $|\mathcal{V}|$.

Each token is assigned an integer ID during tokenizer construction, for example:

$$\text{"the"} \mapsto 464, \quad \text{"cat"} \mapsto 3797, \quad \text{"sat"} \mapsto 3332$$

Special tokens are also included: `[BOS]` (beginning of sequence), `[EOS]` (end of sequence), `[PAD]` (padding), `[UNK]` (unknown).

A sentence is encoded as a sequence of IDs, forming the input to the embedding layer:

$$\text{"The cat sat"} \mapsto [464,\ 3797,\ 3332]$$

Token Embeddings

Token IDs are discrete integers. A **token embedding** maps each ID to a dense real-valued vector:

$$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$$

The embedding matrix $\mathbf{E}$ has one row per vocabulary token. For a token with ID k, the embedding is the k-th row:

$$\mathbf{x}_k = \mathbf{E}[k, :] \in \mathbb{R}^d$$

This is a **linear** lookup — during training, the rows of $\mathbf{E}$ are updated by gradient descent to learn useful vector representations that capture semantic and syntactic relationships between tokens.

The Embedding Matrix

The embedding dimension d is a key hyperparameter — typical values range from $d = 512$ for small research models up to many thousands for very large models.

For an input sequence of n tokens, the embedding layer produces a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{pmatrix}$$

The Ordering Problem

Attention without positional information is **permutation-equivariant**: if the input tokens are reordered, the outputs are reordered in the same way.

For a permutation matrix $\mathbf{P}$:

$$\text{Attention}(\mathbf{PX}) = \mathbf{P}\,\text{Attention}(\mathbf{X})$$

Without additional information, the Transformer cannot tell whether token i appears before or after token j.

Solution: add a **positional encoding** vector to each token embedding before feeding the sequence into the attention layers. This injects order information into the representations.

Sinusoidal Positional Encoding

The original Transformer (Vaswani et al., 2017) uses **deterministic sinusoidal encoding**. For position $pos \in \{0, 1, \ldots, n-1\}$ and embedding dimension index $i \in \{0, 1, \ldots, d/2 - 1\}$:

$$PE(pos,\, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos,\, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- The denominator $10000^{2i/d}$ spans a wide range of frequencies across the d dimensions
- **No parameters are learned** — the encoding is fixed and added element-wise to the token embeddings: $\mathbf{z}_i = \mathbf{x}_i + PE(i, :)$

Sinusoidal Positional Encoding

The same sinusoidal formula is used in diffusion models to encode the **noise timestep** t (as covered in the diffusion models chapter):

$$\mathrm{emb}(t)_{2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \mathrm{emb}(t)_{2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

In both cases, the 10000 is a fixed scale hyperparameter that spreads sinusoidal frequencies across the embedding dimensions. As the dimension index i increases, the denominator $10000^{2i/d}$ increases, resulting in lower-frequency components. Thus, lower dimensions encode fast-changing patterns while higher dimensions encode slower-varying patterns.

Positional Encoding

Absolute positional sinusoidal encoding:

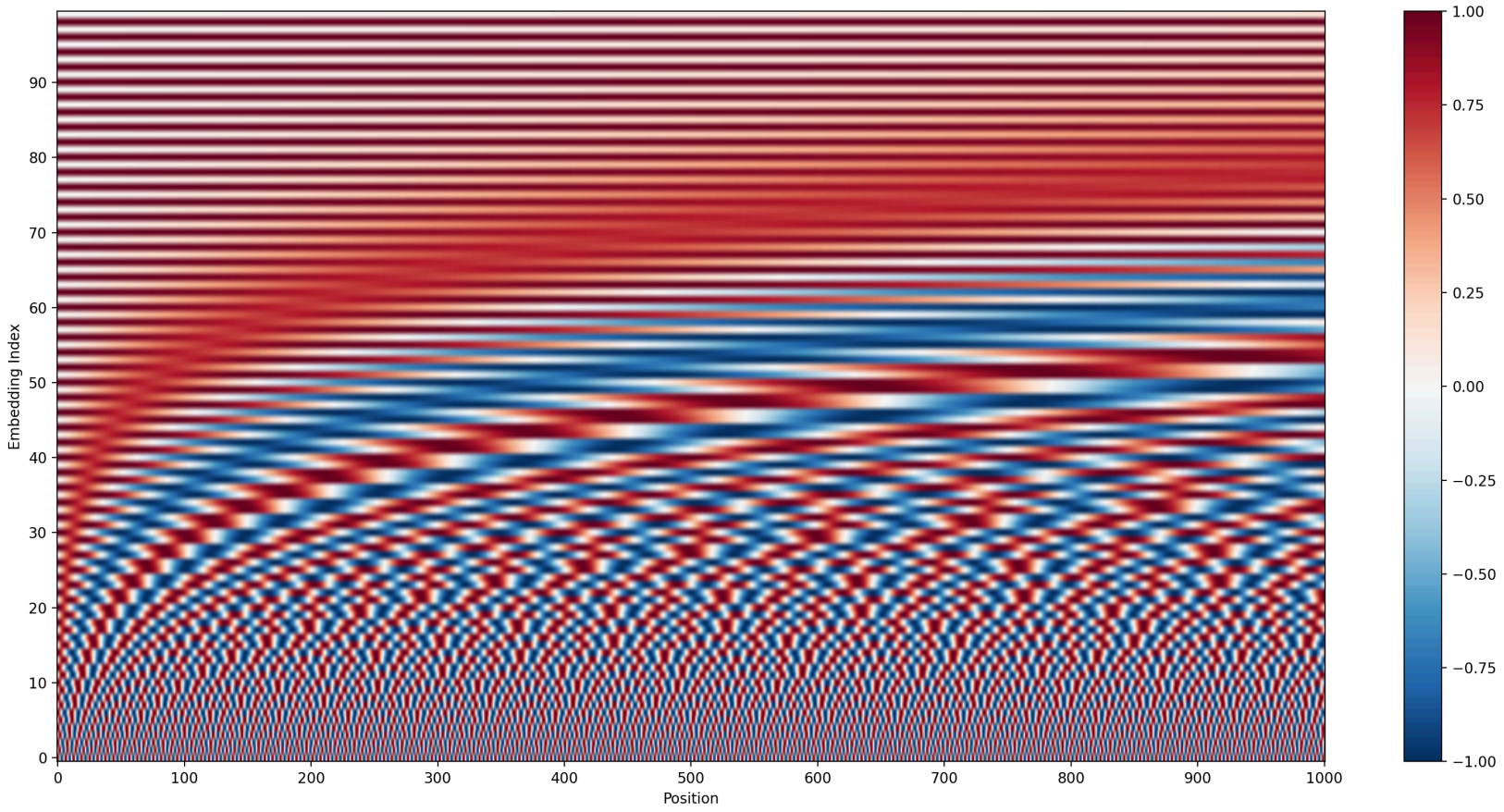


Image source: Nils Blümer, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Absolute_positional_encoding.png

Sinusoidal Encoding Properties

Key properties of sinusoidal positional encoding:

- **Unique over practical sequence lengths**: each position pos receives a distinct encoding vector in $\mathbb{R}^d$
- **Bounded**: all values are in $[-1, 1]$ — no scaling issues regardless of sequence length
- **Can be evaluated beyond training length**: the formula applies to any pos, even positions not seen during training, although performance at significantly longer sequence lengths may degrade due to distribution shift

Transformer Networks

A standard transformer with embedding, positional encoding, encoder-decoder blocks and linear prediction:

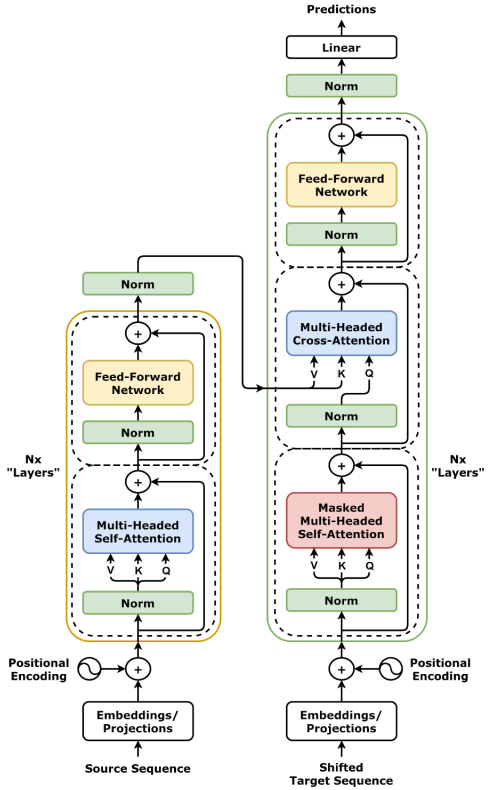


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_full_architecture.png

One Encoder Layer

An encoder layer with self-attention:

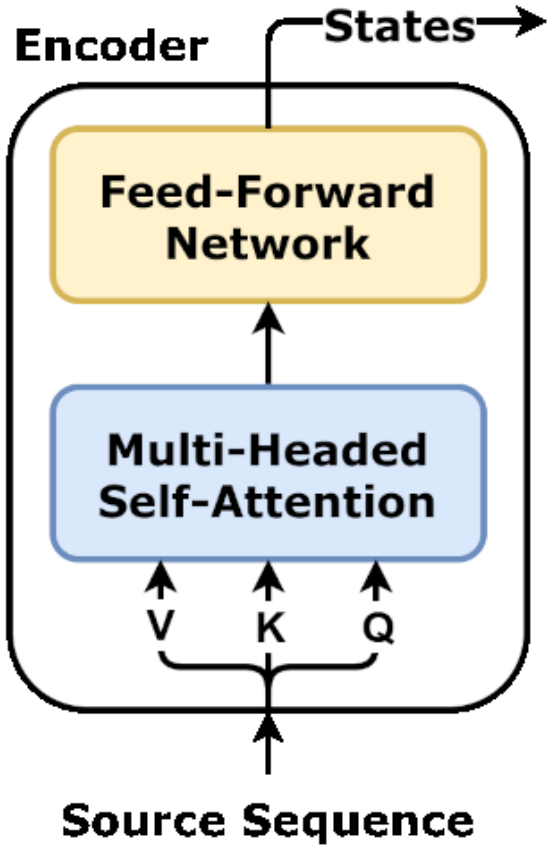


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder_block.png

One Decoder Layer

A decoder layer with masked self-attention and cross-attention:

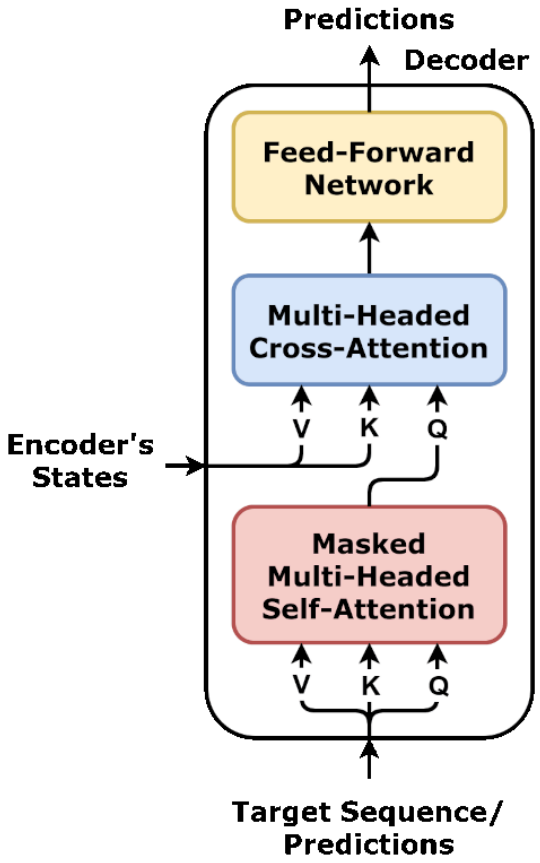


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_decoder_block.png

Encoder-Decoder Block

A single encoder-decoder block with self, cross and masked multi-head attention:

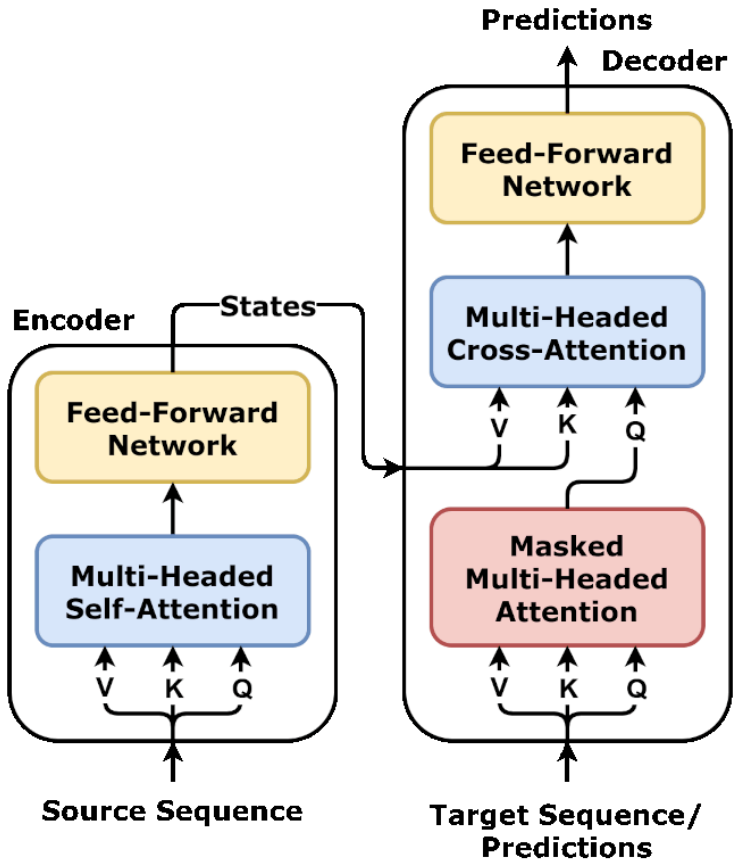


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder-decoder_block.png

Attention Revisited

In the attention chapter, we introduced **scaled dot-product attention** (Vaswani et al., 2017) with query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$ and value matrix $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, $\mathbf{K} \in \mathbb{R}^{m \times d_k}$, $\mathbf{V} \in \mathbb{R}^{m \times d_v}$, where n and m are the number of tokens in the query and key/value sequences, respectively — **each row corresponds to one token**
- The product $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times m}$ is a token-to-token score matrix: entry (i, j) measures how strongly token i attends to token j along the sequence
- **Self-attention**: $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ all originate from the same input sequence via learned projection matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

Self-Attention in the Encoder

In the encoder, every position attends to **all positions** in the input, including itself — no causal masking.

Given positioned input $\mathbf{Z} \in \mathbb{R}^{n \times d}$ (token embeddings with positional encodings added), where each of the n rows of $\mathbf{Z}$ corresponds to one token in the sequence:

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^V$$

$$\mathbf{O} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Since every position has full access to every other position, encoder self-attention is **bidirectional** — making the encoder well-suited for building rich contextual representations of the full input sequence.

Masked Self-Attention in the Decoder

During autoregressive generation the decoder must not attend to future tokens. A **causal mask** is applied before softmax (as introduced in the attention chapter):

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

After softmax, the $-\infty$ entries receive probability 0, so position i only aggregates information from positions up to and including i. This **unidirectional** (causal) attention supports both parallel training via teacher forcing and sequential autoregressive inference.

Attention in a Transformer

A single attention head with scaled dot product attention scores:

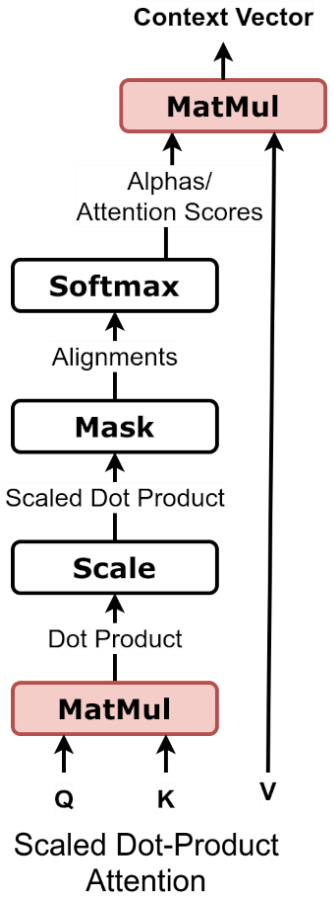


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_attention_block_diagram.png

Multi-Head Attention in a Transformer

Multi-head attention by concatenating independent heads:

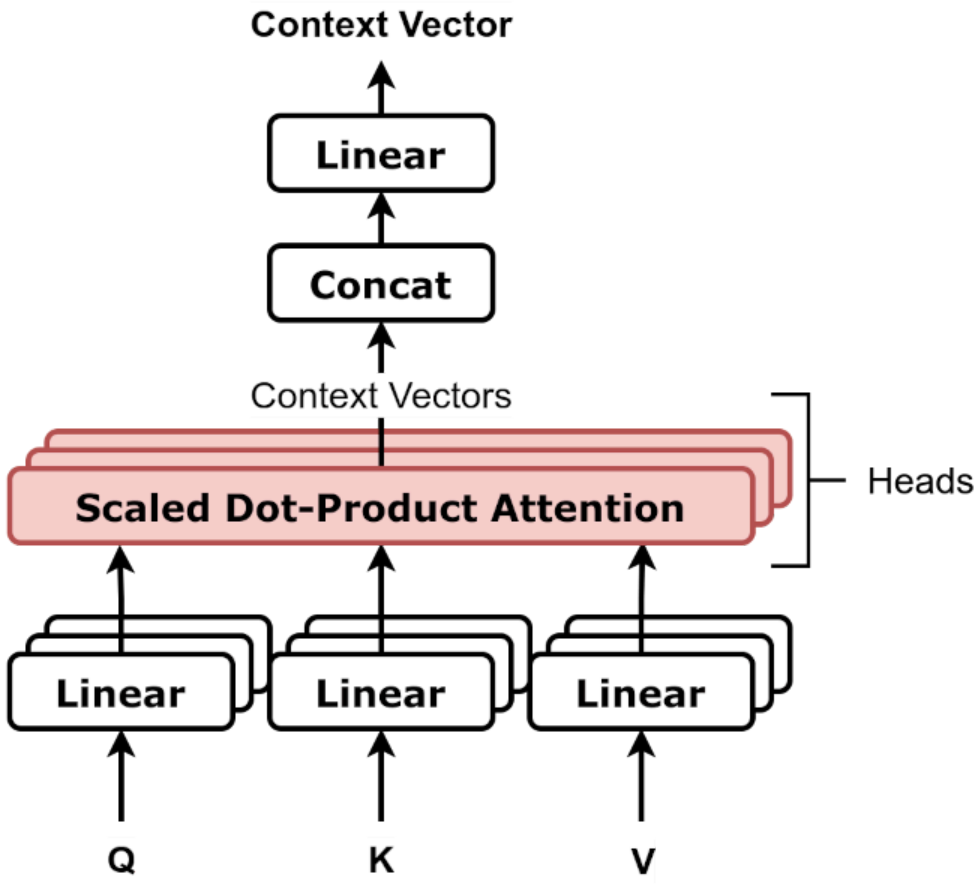


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Multiheaded_attention,_block_diagram.png

Cross-Attention in the Transformer

The decoder uses **cross-attention** to read from the encoder's output. Queries come from the decoder, keys and values come from the encoder:

$$\mathbf{Q} = \mathbf{H}^{(\text{dec})}\mathbf{W}^{Q}, \quad \mathbf{K} = \mathbf{H}^{(\text{enc})}\mathbf{W}^{K}, \quad \mathbf{V} = \mathbf{H}^{(\text{enc})}\mathbf{W}^{V}$$

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{H}^{(\text{dec})}$: decoder hidden states after masked self-attention
- $\mathbf{H}^{(\text{enc})}$: encoder hidden states from the top encoder layer

Each decoder position attends to **all encoder positions**, learning which parts of the source are most relevant for predicting the current target token.

Cross-Attention: Q, K, V Sources

Component	Source
Queries $\mathbf{Q}$	Decoder hidden states
Keys $\mathbf{K}$	Encoder hidden states
Values $\mathbf{V}$	Encoder hidden states

This is the same cross-attention mechanism introduced in the attention chapter — the key–value source is the encoder, while the query source is the decoder.

This allows the decoder to **condition on the full encoded source sequence** at every decoding step.

Encoder-Decoder Block

A single encoder-decoder block with self, cross and masked multi-head attention:

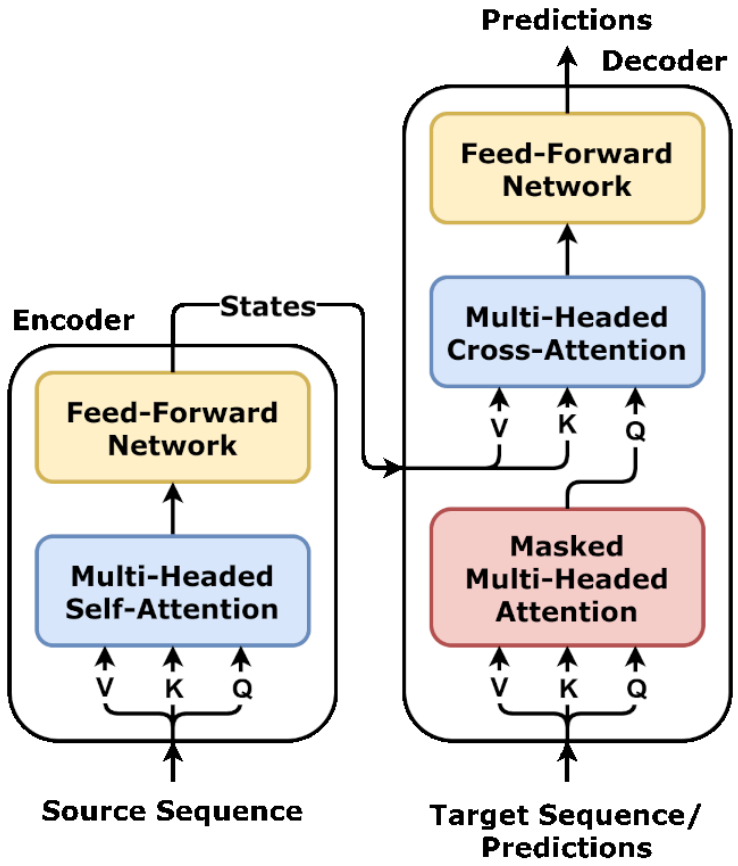


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_one_encoder-decoder_block.png

Feed-Forward Sublayer

Each encoder and decoder layer contains a **position-wise feed-forward network** (FFN), applied **independently** to each sequence position:

$$\mathrm{FFN}(\mathbf{z}) = \mathbf{W}_2\,\sigma(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$$

where σ is a non-linear activation function (ReLU in the original paper).

- $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$ — the inner dimension is typically $d_{\text{ff}} = 4d$
- The FFN introduces **non-linearity** and expands the model's representational capacity
- The same weight matrices are shared across all positions in the sequence

Stacking Encoder and Decoder Layers

A single encoder or decoder layer provides limited representational depth. In the full Transformer, N layers are **stacked**:

- **Encoder stack**: N identical encoder layers applied in sequence; each layer refines the representations with self-attention and FFN
- **Decoder stack**: N identical decoder layers; each applies masked self-attention, cross-attention over the encoder output, and FFN

Stacked Encoder-Decoder Transformers

A single-layer and stacked encoder-decoder transformer side-by-side:

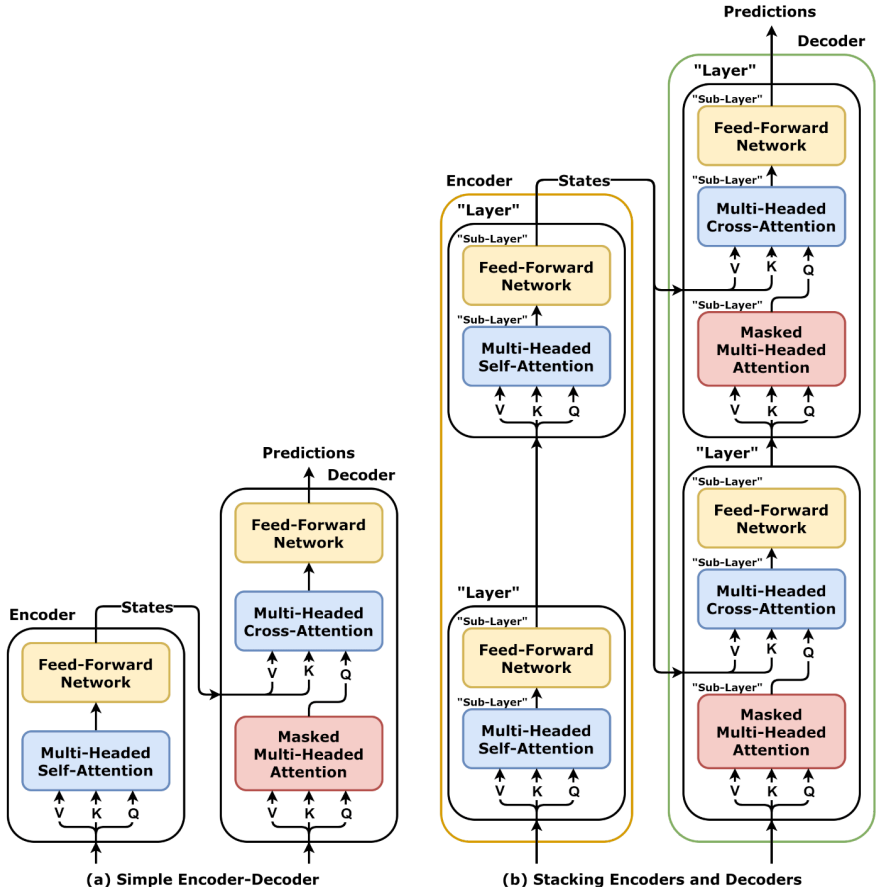


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_stacked_multilayers.png

Transformer Networks

A transformer with multi-layered encoder-decoder networks:

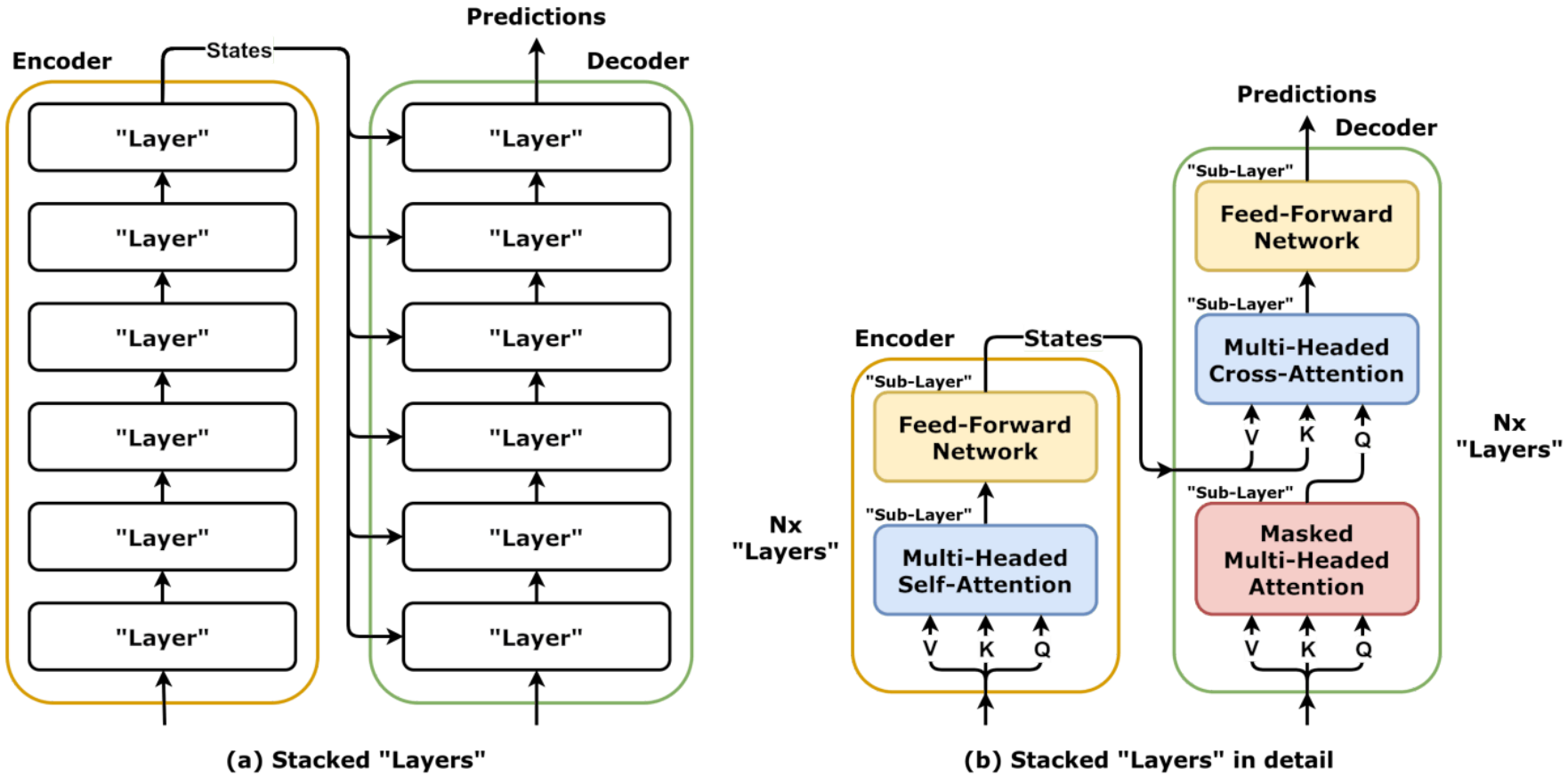


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_stacked_layers_and_sublayers.png

Transformer Networks

A standard transformer with embedding, positional encoding, encoder-decoder blocks and linear prediction:

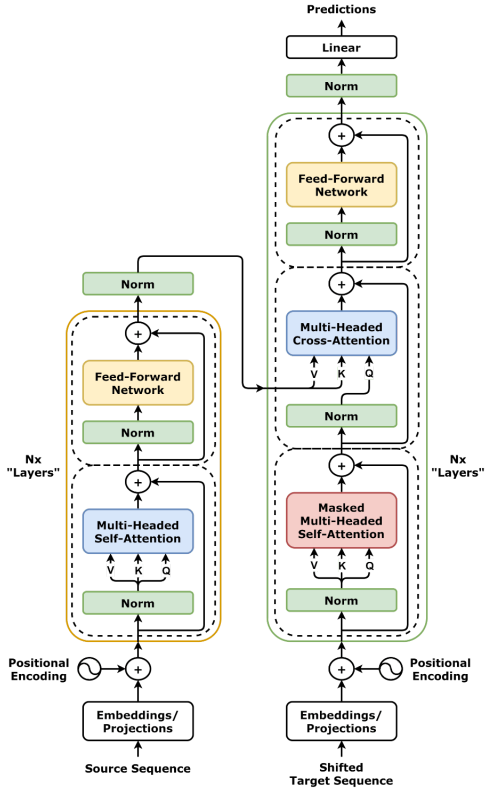


Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_full_architecture.png

Encoder-Only Transformers

Encoder-only transformers use only the encoder stack. All positions attend to all others (bidirectional self-attention):

- Pre-trained with **masked language modeling** — randomly mask a fraction of tokens and predict them from the full context
- Well-suited for **understanding** tasks: classification, named entity recognition
- **Not suitable** for autoregressive generation — the model sees the full sequence at once

Applications: sentiment analysis, semantic search, text classification.

Decoder-Only Transformers

Decoder-only transformers use only the decoder stack with masked (causal) self-attention:

- Pre-trained with **next-token prediction** — predict token $t+1$ given tokens $1, \ldots, t$
- All positions attend only to previous positions — **unidirectional**
- Well-suited for **text generation**: language modeling, code completion, dialogue

At scale, decoder-only models can exhibit **emergent capabilities** such as few-shot learning, chain-of-thought reasoning when prompted appropriately, and in-context learning without weight updates.

Most modern large language models are decoder-only architectures.

GPT is Decoder-Only

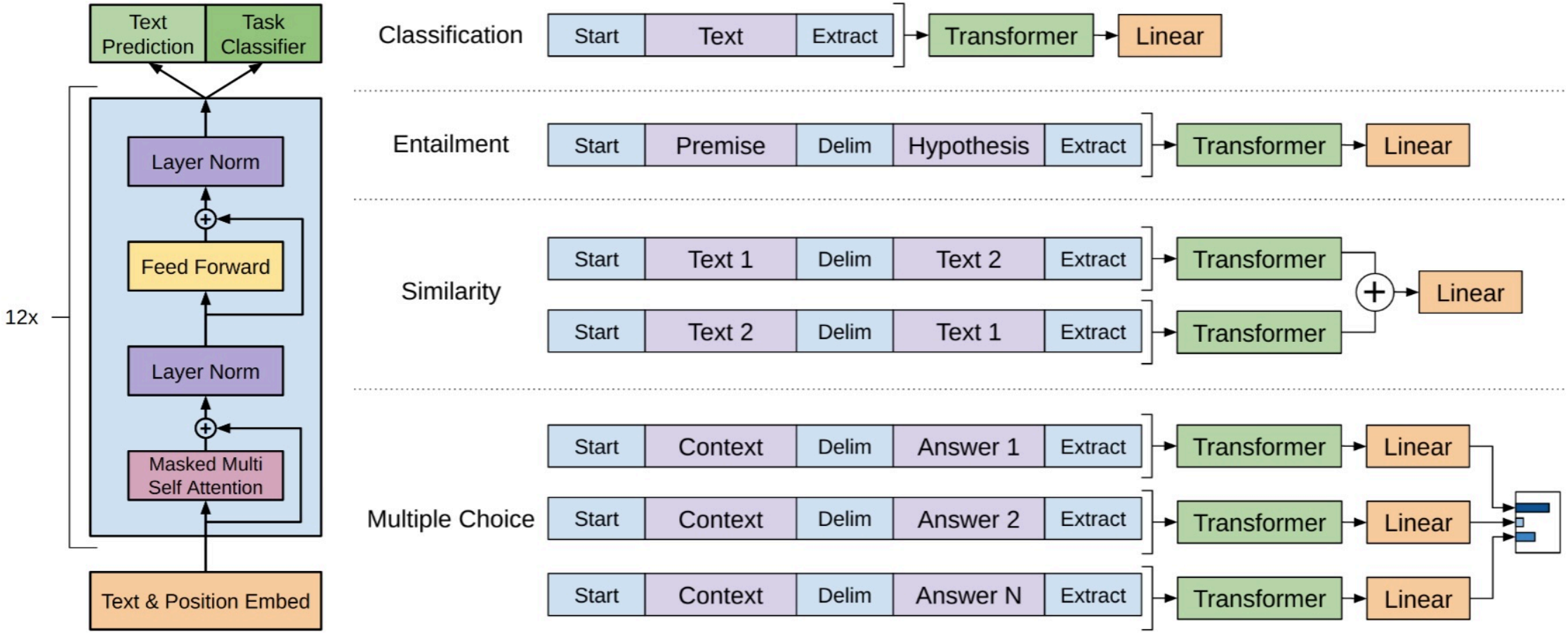


Image source: Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training.

Encoder-Decoder (Seq2Seq) Transformers

Encoder-decoder transformers use both stacks — the architecture introduced by Vaswani et al. (2017):

Well-suited for **sequence-to-sequence** tasks: machine translation, summarization, question generation.

The encoder processes the full source with bidirectional attention; the decoder generates the target autoregressively, attending to the encoder output via cross-attention at every step.

Autoregressive Language Modeling

The model predicts token t_i from a prefix representation $\mathbf{h}_{<i}$ computed from tokens $t_0, \ldots, t_{i-1}$:

$$\hat{P}(t_i \mid t_0, \ldots, t_{i-1}) = \mathrm{softmax}(\mathbf{W}_{\mathrm{out}}\, \mathbf{h}_{<i})$$

where $\mathbf{h}_{<i}$ explicitly denotes the representation of the prefix before token t_i, and $\mathbf{W}_{\mathrm{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}$

Training a Transformer

In language models, the target sequence is the input sequence shifted by one token:

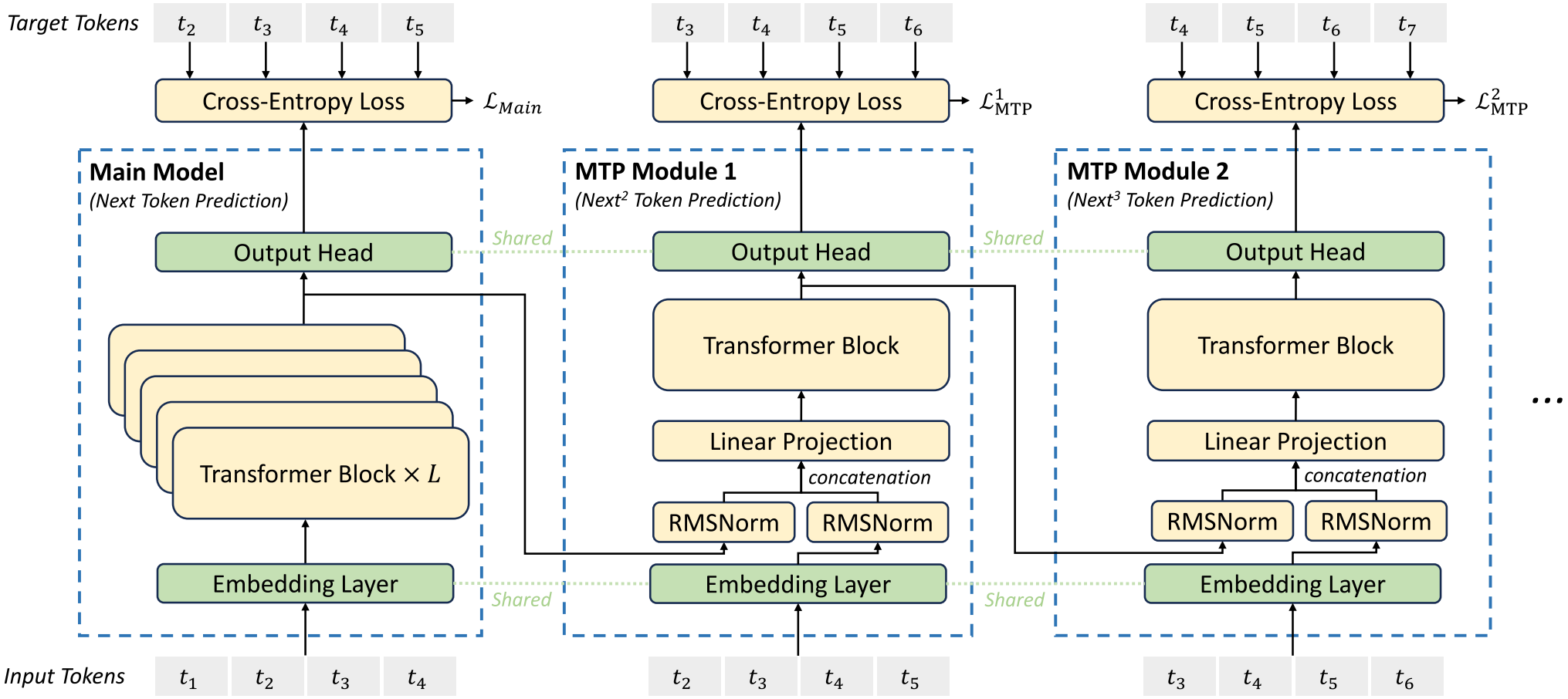


Image source: DeepSeek, MIT , via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Multi-Token_Prediction_(DeepSeek)_01.svg

Teacher Forcing

During training, the decoder receives the **ground-truth** tokens as input at each step — not its own previous predictions. This is called **teacher forcing**:

- At training step i, the decoder input is the ground-truth token t_{i-1}
- The decoder predicts a probability distribution over $\mathcal{V}$ for position i
- All positions are trained **in parallel** using the causal mask — enabling efficient GPU utilization

Teacher Forcing

At **inference** time, the model generates autoregressively:

1. Feed the start token `[BOS]`
2. Predict $\hat{t}_2 = \arg\max \hat{P}(\cdot \mid t_1)$ (or sample stochastically)
3. Append $\hat{t}_2$ and predict $\hat{t}_3$
4. Repeat until `[EOS]` or maximum length

Teacher forcing enables efficient parallel training while maintaining the autoregressive structure at inference.

Training Loss: Cross-Entropy

The training objective is to maximize the log-likelihood of the correct next token at every position. With a start token $t_0 = [\mathrm{BOS}]$ and ground-truth tokens $t_1, \ldots, t_n$, this is equivalent to minimizing the **cross-entropy loss**:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \log \hat{P}(t_i \mid t_0, \ldots, t_{i-1})$$

For a vocabulary of size $|\mathcal{V}|$, this can be written as a standard cross-entropy between the predicted distribution $\hat{\mathbf{y}}_i \in \mathbb{R}^{|\mathcal{V}|}$ and the one-hot target $\mathbf{y}_i$:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \mathbf{y}_i^\top \log \hat{\mathbf{y}}_i$$

LLaMa2 Training

Training loop of pretraining, finetuning and reinforcement learning in LLaMa2:

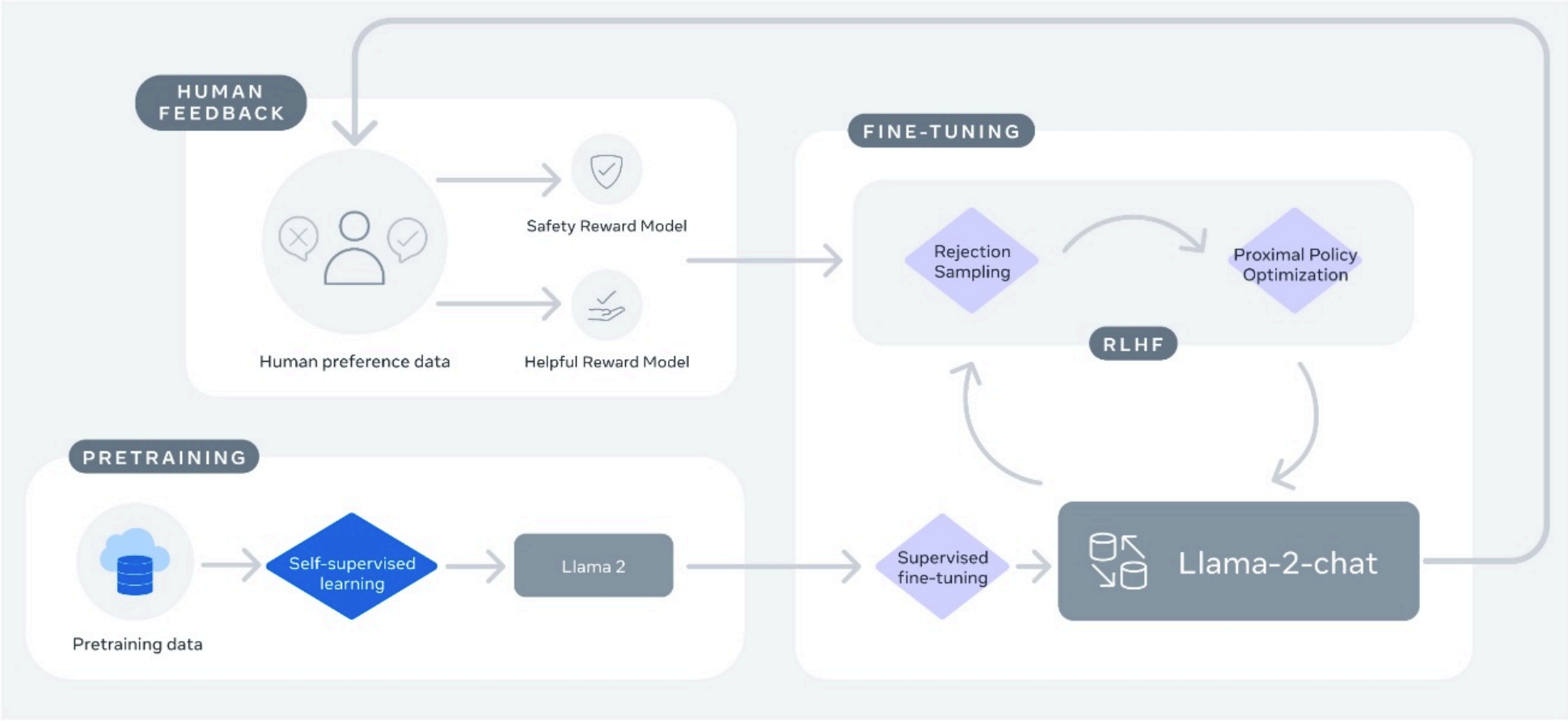


Image source: Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288.

Training at Scale

Large language models are trained on massive corpora with billions to trillions of tokens. Model sizes range from hundreds of millions to hundreds of billions of parameters.

Scaling laws (Kaplan et al., 2020) show that loss decreases as a predictable power law in both model size and dataset size — motivating ever-larger training runs.

Large-batch distributed training across hundreds or thousands of GPUs is standard.

Training at Scale

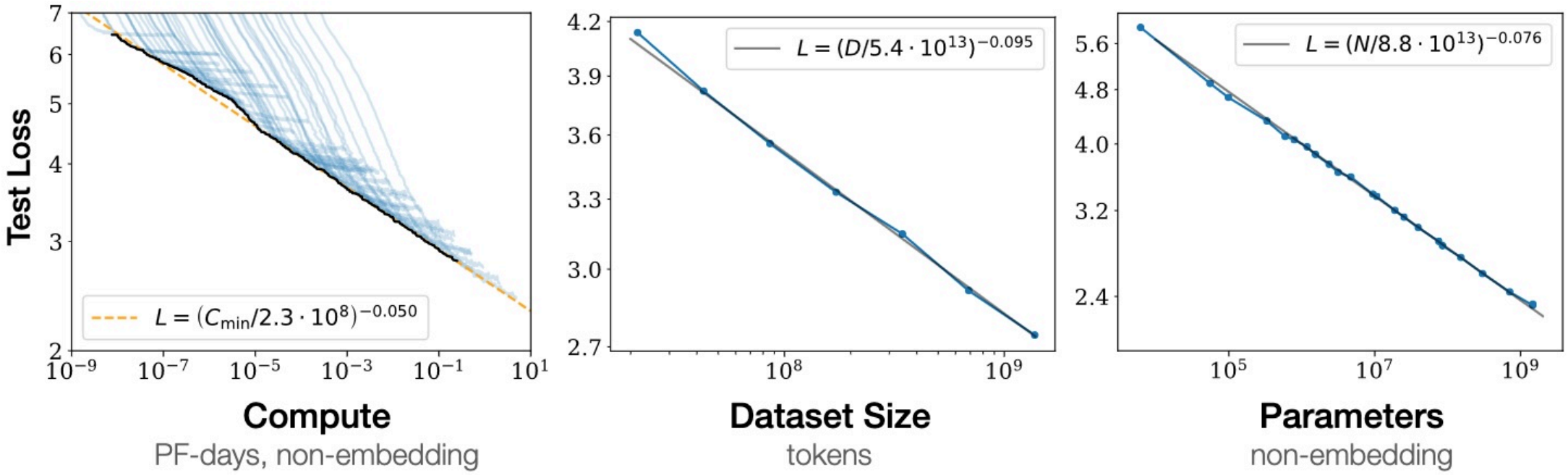


Image source: Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

The Context Window

The **context window** (or **context length**) is the maximum number of tokens the model can process in a single forward pass — the maximum value of n.

Everything the model can "see" and reason about at inference time must fit within this window:

- The system prompt
- The conversation history
- The current user input
- Any retrieved documents

If the input exceeds the context window, some tokens must be truncated, summarized or retrieved separately — tokens outside the current window are not processed in that forward pass.

Context Window: Memory and Compute

For a full-sequence forward pass, self-attention computes an $n \times n$ score matrix, so the attention score memory and compute costs grow **quadratically** with context length n:

$$\text{Attention score matrix size} = n^2 \text{ elements per head}$$

A model with a **16k context window** can attend to twice as many tokens as an 8k model, which requires approximately **four times** the memory for attention scores.

Context Window: Practical Meaning

What 8k vs. 16k means in practice:

Task	8k context	16k context
Short-document Q&A	✅ Fits easily	✅ Fits easily
Long article summarization	⚠️ May truncate	✅ Usually fits
Full book processing	❌ Requires chunking	❌ Requires chunking
Multi-turn dialogue	⚠️ History lost quickly	✅ Longer history preserved

The supported context window is determined by the architecture, positional encoding and training setup. Learned absolute positional embeddings impose a hard table size, while sinusoidal encodings can be evaluated at arbitrary positions but do not by themselves guarantee that models generalize to sequence lengths significantly beyond those seen during training.

Summary: The Transformer Architecture

The full Transformer pipeline from raw text to output prediction:

1. **Tokenize** raw text → sequence of token IDs using BPE or similar subword algorithm
2. **Embed** each token ID → dense vector via the embedding matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$
3. **Add positional encoding** → sinusoidal $PE(pos, i)$ added element-wise to each token embedding
4. **Encoder** (if present) → stack of bidirectional self-attention + FFN layers; produces key-value representations for the decoder
5. **Decoder** → stack of masked self-attention + cross-attention + FFN layers; generates output tokens autoregressively
6. **Output projection** → linear layer $\mathbf{W}_{\text{out}}$ + softmax → next-token probability distribution over $\mathcal{V}$

Summary: Why Transformers Work

Challenge	Transformer solution
Sequential bottleneck of RNNs	Parallel self-attention over all positions simultaneously
Vanishing gradients over long sequences	Direct attention connections between any two positions
Fixed-size context bottleneck in RNN seq2seq	Full encoder output accessible at every decoder step via cross-attention
Lack of positional information	Sinusoidal positional encoding $PE(pos, i)$ added to embeddings
Single attention pattern insufficient	Multi-head attention with H independent learned subspaces
Deep network training instability	Residual connections + layer normalization at every sublayer